

Détecteur de Spam AT&T

Description de l'entreprise

AT&T Inc. est une multinationale américaine spécialisée dans les télécommunications, dont le siège social se trouve à la Whitacre Tower, dans le centre-ville de Dallas, au Texas. C'est la plus grande entreprise de télécommunications au monde en termes de chiffre d'affaires et le troisième opérateur de téléphonie mobile aux États-Unis. En 2022, AT&T figurait au 13e rang du classement Fortune 500 des plus grandes entreprises américaines, avec un chiffre d'affaires de 168,8 milliards de dollars.

Problématique

L'un des principaux problèmes rencontrés par les utilisateurs d'AT&T est l'exposition constante aux messages indésirables (spams). AT&T a pu pendant un certain temps signaler manuellement ces messages, mais l'entreprise recherche désormais un moyen automatisé de détecter les spams afin de protéger ses utilisateurs.

Objectif

Construire un détecteur de spam capable de signaler automatiquement les messages indésirables dès leur réception, en se basant uniquement sur le contenu du SMS.

Plan du notebook

1. Installation des dépendances
 2. Importation des bibliothèques
 3. Chargement et exploration des données
 4. Prétraitement des données
 5. Modèle 1 - Réseau de neurones simple avec Embedding + BiLSTM
 6. Modèle 2 - Transfert d'apprentissage avec DistilBERT
 7. Comparaison des performances et conclusion
-

1. Installation des dépendances

Exécuter cette cellule en premier si les bibliothèques ne sont pas encore installées. Redémarrer le kernel après l'installation, puis continuer l'exécution du notebook.

```
In [1]: # Suppression des warnings de dépréciation TensorFlow pour une sortie plus lisible.
# TF_CPP_MIN_LOG_LEVEL = '3' masque les messages de niveau INFO, WARNING et ERROR, ne laissant que les messages de niveau FATAL.
# Le logger Python de TensorFlow est également limité au niveau ERROR uniquement.
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import logging
logging.getLogger('tensorflow').setLevel(logging.ERROR)

# Installation des bibliothèques nécessaires au projet.
# À exécuter une seule fois, puis redémarrer le kernel.
# tf_keras est requis quand TF_USE_LEGACY_KERAS=True est défini dans l'environnement.

import sys
!{sys.executable} -m pip install tensorflow tf_keras transformers keras_preprocessing \
    scikit-learn pandas numpy matplotlib seaborn --quiet

print("Installation terminée. Redémarrer le kernel puis exécuter les cellules suivantes.")
```

Installation terminée. Redémarrer le kernel puis exécuter les cellules suivantes.

2. Importation des bibliothèques

```
In [2]: # --- Bibliothèques standard ---
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
warnings.filterwarnings("ignore")

# --- Scikit-Learn : prétraitement et évaluation ---
```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    f1_score,
    roc_auc_score
)
from sklearn.preprocessing import LabelEncoder

# --- TensorFlow ---
import tensorflow as tf

# Importation de tf_keras en priorité pour garantir la cohérence de l'API.
# Keras 3 (standalone, disponible depuis TF 2.16) et tf_keras ont des interfaces
# internes incompatibles (callbacks, couches). En forçant tf_keras, tous les
# composants (modèle, callbacks, optimiseurs) partagent la même implémentation.
import tf_keras as keras
from tf_keras import layers
from tf_keras.callbacks import EarlyStopping, ReduceLROnPlateau
print(f"tf_keras version : {keras.__version__}")

# --- Tokenizer et pad_sequences ---
try:
    from keras_preprocessing.text import Tokenizer
    from keras_preprocessing.sequence import pad_sequences
except ModuleNotFoundError:
    from tensorflow.keras.preprocessing.text import Tokenizer
    from tensorflow.keras.preprocessing.sequence import pad_sequences

# --- Transformers HuggingFace : transfert d'apprentissage ---
from transformers import (
    DistilBertTokenizerFast,
    TFDistilBertModel
)

# --- Graine aléatoire pour la reproductibilité ---
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print(f"TensorFlow version : {tf.__version__}")
print("Importations effectuées avec succès.")

```

WARNING:tensorflow:From c:\Users\aida\anaconda3\envs\nlp_env\lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

tf_keras version : 2.18.0

TensorFlow version : 2.18.0

Importations effectuées avec succès.

3. Chargement et exploration des données

```
In [3]: # Chargement du fichier CSV fourni avec Le projet.  
# Le fichier contient 5 colonnes : 'v1' (label), 'v2' (message) et 3 colonnes parasites quasi-vides.  
# L'encodage latin-1 est requis car le fichier contient des caractères non-UTF-8.
```

```
df = pd.read_csv('spam.csv', encoding='latin-1')  
  
print(f"Dimensions brutes : {df.shape}")  
print(f"Colonnes détectées : {df.columns.tolist()}")  
df.head(10)
```

Dimensions brutes : (5572, 5)

Colonnes détectées : ['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']

```
Out[3]:
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN
5	spam	FreeMsg Hey there darling it's been 3 week's n...	NaN	NaN	NaN
6	ham	Even my brother is not like to speak with me. ...	NaN	NaN	NaN
7	ham	As per your request 'Melle Melle (Oru Minnamin...	NaN	NaN	NaN
8	spam	WINNER!! As a valued network customer you have...	NaN	NaN	NaN
9	spam	Had your mobile 11 months or more? U R entitle...	NaN	NaN	NaN

```
In [4]: # Nettoyage du DataFrame : conservation des deux colonnes utiles et renommage.  
# Les colonnes 'Unnamed: 2/3/4' sont quasi-vides (moins de 1 % de valeurs renseignées)  
# et ne contiennent pas d'information exploitable pour la classification.
```

```
df = df[['v1', 'v2']].copy()  
df.columns = ['label', 'message']  
  
# Suppression des éventuels doublons pour éviter de biaiser l'évaluation  
nb_doublons = df.duplicated().sum()  
df = df.drop_duplicates().reset_index(drop=True)  
  
print(f"Doublons supprimés : {nb_doublons}")  
print(f"Dimensions finales : {df.shape}")  
print(f"Valeurs manquantes : {df.isnull().sum().sum()}")  
df.head()
```

```
Doublons supprimés : 403  
Dimensions finales : (5169, 2)  
Valeurs manquantes : 0
```

```
Out[4]:
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
In [5]: # Distribution des classes spam / ham.  
# Le jeu de données est déséquilibré : environ 87 % de ham et 13 % de spam.  
# Ce déséquilibre sera pris en compte lors de l'évaluation (F1-Score, AUC-ROC).
```

```
print("Distribution des classes :")  
print(df['label'].value_counts())  
print()  
print(f"Proportion de spam : {df['label'].value_counts(normalize=True)['spam']:.2%}")  
print(f"Proportion de ham : {df['label'].value_counts(normalize=True)['ham']:.2%}")
```

Distribution des classes :

label

ham 4516

spam 653

Name: count, dtype: int64

Proportion de spam : 12.63%

Proportion de ham : 87.37%

```
In [6]: # Visualisation de la distribution des classes et de la longueur des messages.  
# Les messages spam sont généralement plus longs que les messages ham,  
# ce qui constitue une première piste discriminante pour les modèles.
```

```
df['longueur'] = df['message'].apply(len)  
  
fig, axes = plt.subplots(1, 3, figsize=(16, 4))  
  
# -- Diagramme en barres : effectifs --  
counts = df['label'].value_counts()  
axes[0].bar(counts.index, counts.values,  
            color=['steelblue', 'tomato'], edgecolor='black', alpha=0.85)  
axes[0].set_title('Effectifs par classe', fontsize=12)  
axes[0].set_xlabel('Classe')  
axes[0].set_ylabel('Nombre de messages')  
for i, v in enumerate(counts.values):  
    axes[0].text(i, v + 30, str(v), ha='center', fontweight='bold')  
  
# -- Diagramme circulaire --  
axes[1].pie(  
    counts.values,  
    labels=counts.index,  
    autopct='%1.1f%%',  
    colors=['steelblue', 'tomato'],  
    startangle=90,  
    explode=(0.05, 0.05)  
)  
axes[1].set_title('Répartition spam / ham', fontsize=12)  
  
# -- Distribution des longueurs par classe --  
for label, color in zip(['ham', 'spam'], ['steelblue', 'tomato']):  
    axes[2].hist(  
        df[df['label'] == label]['longueur'],  
        bins=50, alpha=0.6, label=label, color=color, edgecolor='white'  
    )
```

```

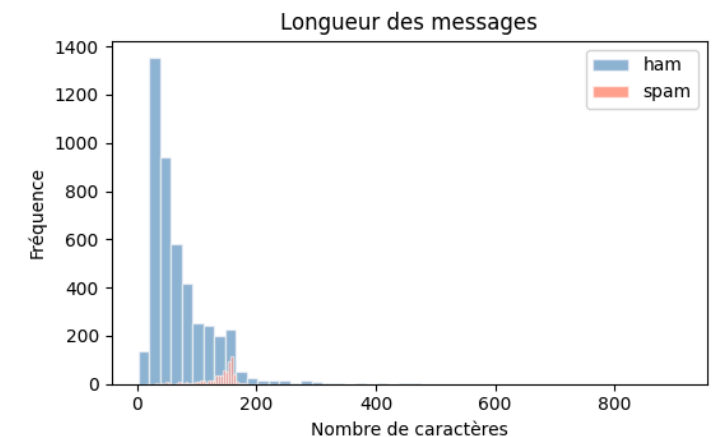
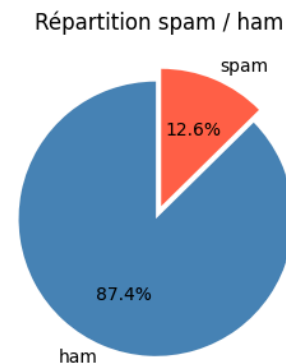
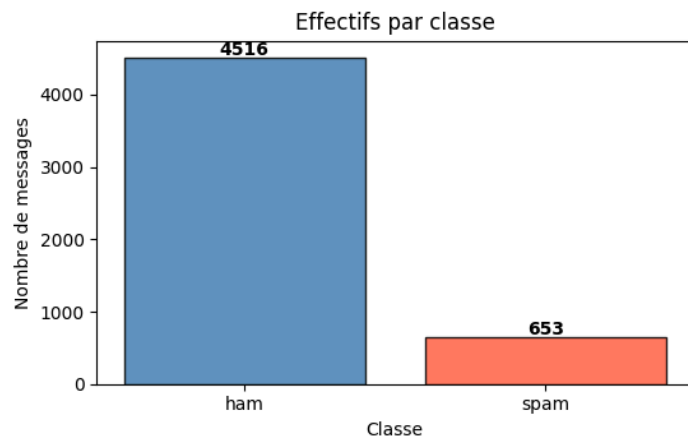
axes[2].set_title('Longueur des messages', fontsize=12)
axes[2].set_xlabel('Nombre de caractères')
axes[2].set_ylabel('Fréquence')
axes[2].legend()

plt.suptitle('Exploration du jeu de données', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print('Statistiques de longueur par classe :')
print(df.groupby('label')['longueur'].describe().round(1))

```

Exploration du jeu de données



Statistiques de longueur par classe :

	count	mean	std	min	25%	50%	75%	max
label								
ham	4516.0	70.5	56.4	2.0	34.0	52.0	90.0	910.0
spam	653.0	137.9	30.1	13.0	132.0	149.0	157.0	224.0

4. Prétraitement des données

In [7]: *# Encodage de la variable cible : 'spam' -> 1, 'ham' -> 0.*
LabelEncoder assigne les valeurs par ordre alphabétique : ham = 0, spam = 1.

```

le = LabelEncoder()
df['label_enc'] = le.fit_transform(df['label'])

```

```
print('Correspondance des labels :')
for cls, enc in zip(le.classes_, le.transform(le.classes_)):
    print(f"  '{cls}' -> {enc}")
```

Correspondance des labels :

```
'ham' -> 0
'spam' -> 1
```

In [8]: *# Nettoyage textuel pour Le Modèle 1 (Embedding + BiLSTM).*
On conserve Les chiffres car ils sont fréquents dans Les spams (numéros courts, montants).
Note : Les textes originaux seront utilisés pour DistilBERT qui gère sa propre tokenisation.

```
def nettoyer_texte(texte):
    """
    Nettoie un SMS pour la tokenisation Keras :
    - Mise en minuscules
    - Suppression des caractères spéciaux non alphanumériques
    - Normalisation des espaces multiples
    """
    texte = texte.lower()
    texte = re.sub(r'^a-z0-9\s', '', texte)
    texte = re.sub(r'\s+', ' ', texte).strip()
    return texte

df['message_clean'] = df['message'].apply(nettoyer_texte)

# Vérification sur des exemples de spam
print('Exemples avant / après nettoyage (spam) :')
for i in df[df['label'] == 'spam'].index[:3]:
    print(f"  Original : {df['message'].iloc[i][:80]}")
    print(f"  Nettoyé   : {df['message_clean'].iloc[i][:80]}")
    print()
```

Exemples avant / après nettoyage (spam) :

```
Original : Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 8
Nettoyé   : free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87
```

```
Original : FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some
Nettoyé   : freemsg hey there darling its been 3 weeks now and no word back id like some fun
```

```
Original : WINNER!! As a valued network customer you have been selected to receivea £900 p
Nettoyé   : winner as a valued network customer you have been selected to receivea 900 prize
```



```
In [9]: # Séparation train / test avec stratification (ratio 80 / 20).
# La stratification garantit que les proportions spam/ham sont identiques
# dans les deux sous-ensembles, ce qui est essentiel avec un jeu déséquilibré.
# On effectue deux splits parallèles :
#   - messages nettoyés pour le Modèle 1
#   - messages originaux pour DistilBERT (Modèle 2)

X_clean = df['message_clean'].values
X_orig = df['message'].values
y = df['label_enc'].values

X_train, X_test, y_train, y_test = train_test_split(
    X_clean, y, test_size=0.2, random_state=SEED, stratify=y
)
X_train_orig, X_test_orig, _, _ = train_test_split(
    X_orig, y, test_size=0.2, random_state=SEED, stratify=y
)

print(f"Taille du jeu d'entraînement : {len(X_train)} exemples")
print(f"Taille du jeu de test : {len(X_test)} exemples")
print(f"Proportion de spam (train) : {y_train.mean():.2%}")
print(f"Proportion de spam (test) : {y_test.mean():.2%}")
```

```
Taille du jeu d'entraînement : 4135 exemples
Taille du jeu de test : 1034 exemples
Proportion de spam (train) : 12.62%
Proportion de spam (test) : 12.67%
```

5. Modèle 1 - Réseau de neurones simple avec Embedding + BiLSTM

Conformément aux recommandations de l'énoncé, nous commençons par **un modèle simple**. La couche d'Embedding apprend des représentations vectorielles des mots directement sur nos données. Le BiLSTM capture les dépendances séquentielles dans les deux sens de lecture.

```
In [10]: # Paramètres de tokenisation.
# VOCAB_SIZE : 10 000 mots Les plus fréquents du corpus (couvre la quasi-totalité des SMS).
# MAX_LEN : 100 tokens - Les SMS dépassent rarement 160 caractères.
# EMBEDDING_DIM : dimension des vecteurs de représentation des mots.

VOCAB_SIZE = 10000
MAX_LEN = 100
```

```
EMBEDDING_DIM = 64
```

```
# Le tokeniseur est ajusté uniquement sur les données d'entraînement  
# pour éviter toute fuite d'information depuis le jeu de test.  
tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token='<OOV>')  
tokenizer.fit_on_texts(X_train)  
  
print(f"Taille du vocabulaire détecté : {len(tokenizer.word_index)} mots")  
print(f"Vocabulaire limité à : {VOCAB_SIZE} mots")
```

Taille du vocabulaire détecté : 8201 mots

Vocabulaire limité à : 10000 mots

```
In [11]: # Conversion des textes en séquences d'entiers puis padding.  
# Chaque mot est remplacé par son indice dans le vocabulaire.  
# padding='post' : Les zéros sont ajoutés en fin de séquence (plus courant pour les LSTM).  
# truncating='post' : Les séquences trop longues sont tronquées par la fin.
```

```
X_train_seq = tokenizer.texts_to_sequences(X_train)  
X_test_seq = tokenizer.texts_to_sequences(X_test)  
  
X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post', truncating='post')  
X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LEN, padding='post', truncating='post')  
  
print(f"Forme des données d'entraînement après padding : {X_train_pad.shape}")  
print(f"Forme des données de test après padding : {X_test_pad.shape}")
```

Forme des données d'entraînement après padding : (4135, 100)

Forme des données de test après padding : (1034, 100)

```
In [12]: # Architecture du Modèle 1 : Embedding -> BiLSTM -> Dense -> Sortie sigmoïde.  
# Le BiLSTM (bidirectionnel) lit la séquence dans les deux sens,  
# ce qui permet de capturer des indices contextuels avant et après chaque mot.  
# Le Dropout réduit le surapprentissage en désactivant aléatoirement des neurones.
```

```
def construire_modele_simple(vocab_size, embedding_dim, max_len):  
    modele = keras.Sequential([  
        # Couche d'embedding : transforme les indices en vecteurs denses  
        layers.Embedding(  
            input_dim=vocab_size,  
            output_dim=embedding_dim,  
            input_length=max_len,  
            name='embedding'  
        ),  
        # BiLSTM : capture les relations séquentielles dans les deux directions
```

```
layers.Bidirectional(  
    layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2),  
    name='bilstm'  
)  
# Couche dense intermédiaire avec activation ReLU  
layers.Dense(32, activation='relu', name='dense_1'),  
layers.Dropout(0.3, name='dropout'),  
# Couche de sortie : probabilité d'être un spam (valeur entre 0 et 1)  
layers.Dense(1, activation='sigmoid', name='sortie')  
)  
return modele
```

```
modele1 = construire_modele_simple(VOCAB_SIZE, EMBEDDING_DIM, MAX_LEN)
```

```
# Compilation : optimiseur Adam, entropie croisée binaire pour la classification binaire
```

```
modele1.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

```
modele1.summary()
```

WARNING:tensorflow:From c:\Users\aicha\anaconda3\envs\nlp_env\lib\site-packages\tf_keras\src\layers\rnn\lstm.py:149: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

WARNING:tensorflow:From c:\Users\aicha\anaconda3\envs\nlp_env\lib\site-packages\tf_keras\src\optimizers__init__.py:317: The name tf.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 64)	640000
bilstm (Bidirectional)	(None, 128)	66048
dense_1 (Dense)	(None, 32)	4128
dropout (Dropout)	(None, 32)	0
sortie (Dense)	(None, 1)	33

=====
Total params: 710209 (2.71 MB)
Trainable params: 710209 (2.71 MB)
Non-trainable params: 0 (0.00 Byte)
=====

```
In [13]: # Définition des callbacks pour optimiser l'entraînement.
# EarlyStopping      : arrête si la perte de validation ne progresse plus (patience=3 époques)
#                  et restaure automatiquement les meilleurs poids.
# ReduceLROnPlateau : réduit le taux d'apprentissage de moitié en cas de stagnation,
#                  ce qui aide le modèle à converger vers un meilleur minimum local.

callbacks_m1 = [
    EarlyStopping(
        monitor='val_loss',
        patience=3,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=2,
        verbose=1
    )
]
```

```

    )
]

print('Entraînement du Modèle 1 (Embedding + BiLSTM)...')
historique1 = modele1.fit(
    X_train_pad, y_train,
    epochs=15,
    batch_size=32,
    validation_split=0.15,
    callbacks=callbacks_m1,
    verbose=1
)

```

Entraînement du Modèle 1 (Embedding + BiLSTM)...

Epoch 1/15

WARNING:tensorflow:From c:\Users\aida\anaconda3\envs\nlp_env\lib\site-packages\tf_keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From c:\Users\aida\anaconda3\envs\nlp_env\lib\site-packages\tf_keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

110/110 [=====] - 17s 103ms/step - loss: 0.2433 - accuracy: 0.9189 - val_loss: 0.0592 - val_accuracy: 0.9839 - lr: 0.0010

Epoch 2/15

110/110 [=====] - 10s 88ms/step - loss: 0.0388 - accuracy: 0.9900 - val_loss: 0.0519 - val_accuracy: 0.9871 - lr: 0.0010

Epoch 3/15

110/110 [=====] - 10s 89ms/step - loss: 0.0163 - accuracy: 0.9957 - val_loss: 0.0523 - val_accuracy: 0.9887 - lr: 0.0010

Epoch 4/15

110/110 [=====] - ETA: 0s - loss: 0.0076 - accuracy: 0.9983

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

110/110 [=====] - 10s 91ms/step - loss: 0.0076 - accuracy: 0.9983 - val_loss: 0.0575 - val_accuracy: 0.9871 - lr: 0.0010

Epoch 5/15

110/110 [=====] - 10s 87ms/step - loss: 0.0022 - accuracy: 0.9997 - val_loss: 0.0599 - val_accuracy: 0.9887 - lr: 5.0000e-04

Epoch 5: early stopping

Restoring model weights from the end of the best epoch: 2.

In [14]: *# Visualisation des courbes d'apprentissage du Modèle 1.*
Une courbe de validation qui suit la courbe d'entraînement indique une bonne généralisation.
Un écart croissant entre les deux signifierait du surapprentissage (overfitting).

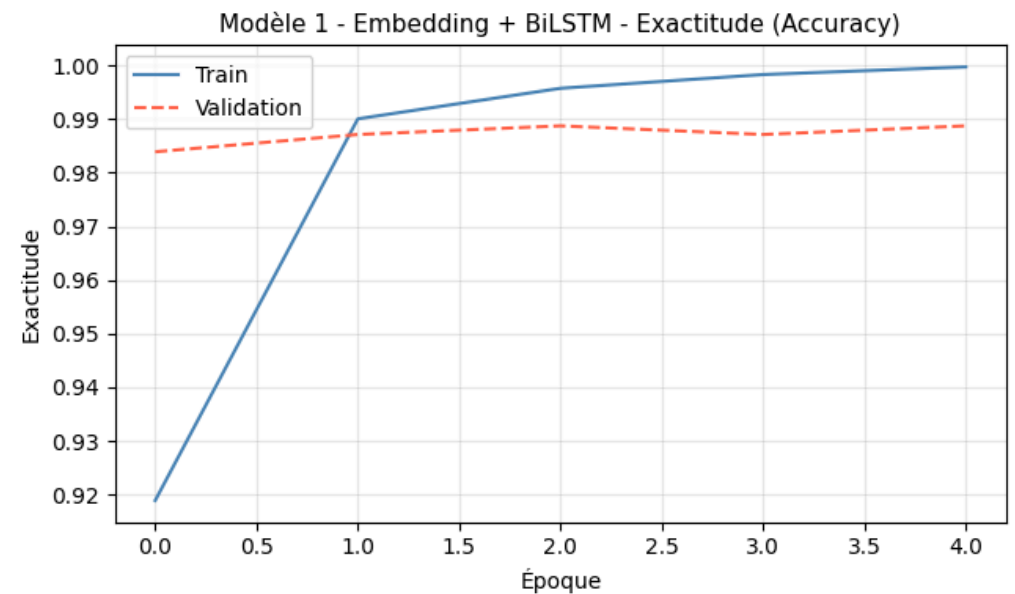
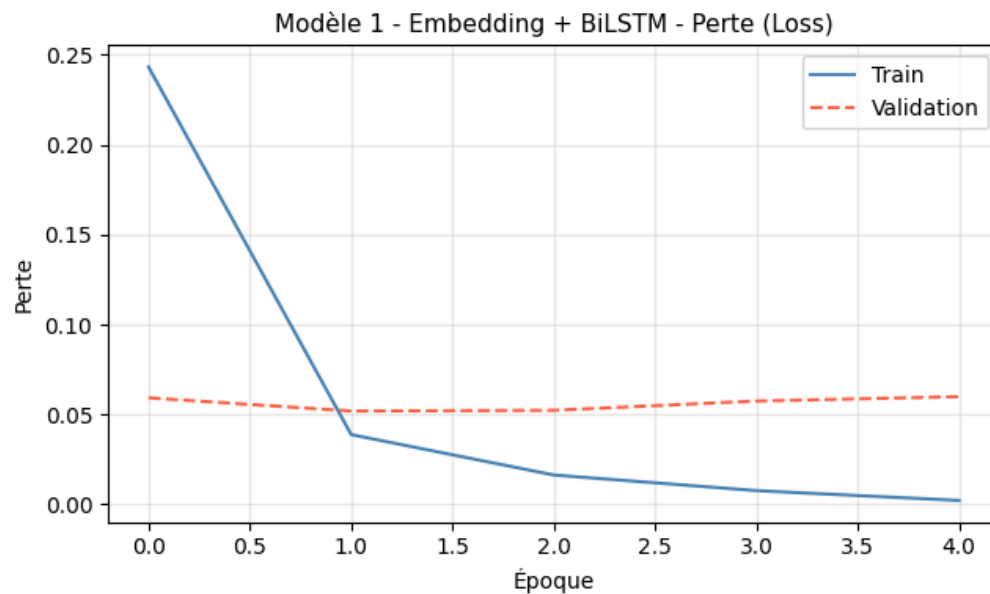
```
def tracer_historique(historique, titre):
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))

    axes[0].plot(historique.history['loss'], label='Train', color='steelblue')
    axes[0].plot(historique.history['val_loss'], label='Validation',
                  color='tomato', linestyle='--')
    axes[0].set_title(f'{titre} - Perte (Loss)', fontsize=11)
    axes[0].set_xlabel('Époque')
    axes[0].set_ylabel('Perte')
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    axes[1].plot(historique.history['accuracy'], label='Train', color='steelblue')
    axes[1].plot(historique.history['val_accuracy'], label='Validation',
                  color='tomato', linestyle='--')
    axes[1].set_title(f'{titre} - Exactitude (Accuracy)', fontsize=11)
    axes[1].set_xlabel('Époque')
    axes[1].set_ylabel('Exactitude')
    axes[1].legend()
    axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.show()

tracer_historique(historique1, 'Modèle 1 - Embedding + BiLSTM')
```



```
In [15]: # Évaluation du Modèle 1 sur Le jeu de test.
# Trois métriques complémentaires sont calculées :
# - Accuracy : taux global de classification correcte
# - F1-Score : moyenne harmonique précision/rappel, adaptée aux classes déséquilibrées
# - AUC-ROC : capacité discriminante, indépendante du seuil de décision

y_pred_proba1 = modele1.predict(X_test_pad).flatten()
y_pred1 = (y_pred_proba1 >= 0.5).astype(int)

acc1 = accuracy_score(y_test, y_pred1)
f1_1 = f1_score(y_test, y_pred1)
roc1 = roc_auc_score(y_test, y_pred_proba1)

print('=' * 52)
print('Résultats - Modèle 1 (Embedding + BiLSTM)')
print('=' * 52)
print(f' Exactitude (Accuracy) : {acc1:.4f}')
print(f' Score F1 (spam) : {f1_1:.4f}')
print(f' AUC-ROC : {roc1:.4f}')
print()
print('Rapport de classification détaillé :')
print(classification_report(y_test, y_pred1, target_names=['ham', 'spam']))
```

```
33/33 [=====] - 1s 12ms/step
```

```
=====
```

```
Résultats - Modèle 1 (Embedding + BiLSTM)
```

```
=====
```

```
Exactitude (Accuracy) : 0.9836
```

```
Score F1 (spam) : 0.9333
```

```
AUC-ROC : 0.9840
```

```
Rapport de classification détaillé :
```

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	903
spam	0.96	0.91	0.93	131
accuracy			0.98	1034
macro avg	0.97	0.95	0.96	1034
weighted avg	0.98	0.98	0.98	1034

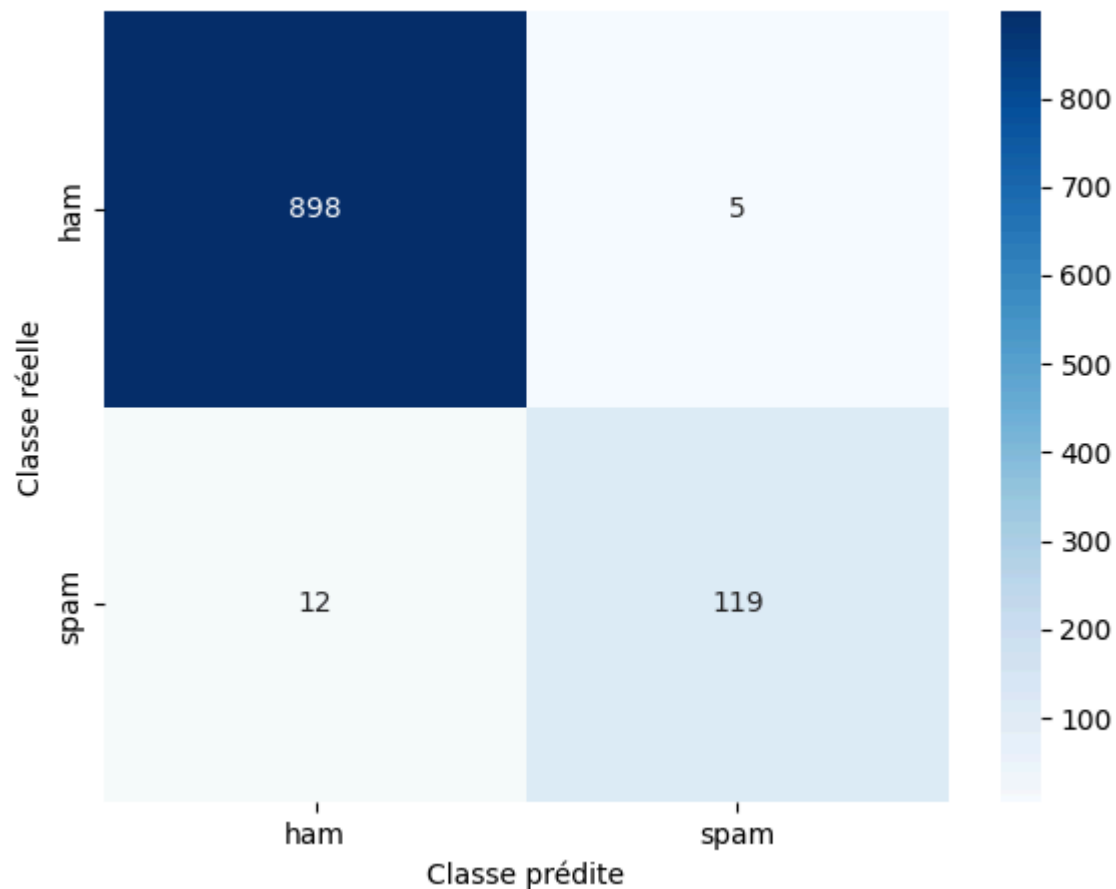
```
In [16]: # Matrice de confusion du Modèle 1.
# Lecture :
```

```
# - Vrais négatifs (haut gauche) : ham correctement identifiés
# - Faux positifs (haut droite) : ham classifiés comme spam (désagrément utilisateur)
# - Faux négatifs (bas gauche) : spam non détectés (risque principal)
# - Vrais positifs (bas droite) : spam correctement bloqués
```

```
def afficher_matrice_confusion(y_true, y_pred, titre):
    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(6, 5))
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=['ham', 'spam'],
        yticklabels=['ham', 'spam'],
        ax=ax
    )
    ax.set_title(f'Matrice de confusion - {titre}', fontsize=12, pad=12)
    ax.set_xlabel('Classe prédite')
    ax.set_ylabel('Classe réelle')
    plt.tight_layout()
    plt.show()

afficher_matrice_confusion(y_test, y_pred1, 'Modèle 1 - Embedding + BiLSTM')
```


Matrice de confusion - Modèle 1 - Embedding + BiLSTM



6. Modèle 2 - Transfert d'apprentissage avec DistilBERT

Conformément aux recommandations de l'énoncé, nous appliquons le **transfert d'apprentissage** avec DistilBERT, une version allégée de BERT pré-entraîné sur des milliards de phrases en anglais. Les poids du modèle de base sont gelés : seule la tête de classification est entraînée sur nos données. Les messages originaux (non nettoyés) sont utilisés car DistilBERT gère sa propre normalisation interne.

```
In [17]: # Chargement du tokeniseur DistilBERT.  
# Le tokeniseur WordPiece découpe les mots en sous-mots et ajoute  
# Les tokens spéciaux [CLS] (début de séquence) et [SEP] (séparateur).  
# MAX_LEN_BERT = 128 : compromis entre couverture du texte et vitesse d'entraînement.
```

```

BERT_MODEL_NAME = 'distilbert-base-uncased'
MAX_LEN_BERT    = 128

bert_tokenizer = DistilBertTokenizerFast.from_pretrained(BERT_MODEL_NAME)

print(f'Tokeniseur chargé : {BERT_MODEL_NAME}')
print(f'Longueur maximale des séquences : {MAX_LEN_BERT} tokens')

```

Tokeniseur chargé : distilbert-base-uncased
Longueur maximale des séquences : 128 tokens

```

In [18]: # Tokenisation des jeux d'entraînement et de test au format DistilBERT.
# input_ids      : indices des sous-mots dans le vocabulaire BERT (30 522 entrées)
# attention_mask : masque binaire distinguant les vrais tokens (1) du padding (0)

def tokeniser_bert(textes, tokenizer, max_len):
    """
    Tokenise une liste de textes pour DistilBERT.
    Retourne les input_ids et attention_masks sous forme de tenseurs TensorFlow.
    """
    encodage = tokenizer(
        list(textes),
        max_length=max_len,
        padding='max_length',
        truncation=True,
        return_tensors='tf'
    )
    return encodage['input_ids'], encodage['attention_mask']

print('Tokenisation en cours...')
train_ids, train_masks = tokeniser_bert(X_train_orig, bert_tokenizer, MAX_LEN_BERT)
test_ids,  test_masks  = tokeniser_bert(X_test_orig,  bert_tokenizer, MAX_LEN_BERT)

print(f'Forme input_ids (train) : {train_ids.shape}')
print(f'Forme input_ids (test)  : {test_ids.shape}')

```

Tokenisation en cours...
Forme input_ids (train) : (4135, 128)
Forme input_ids (test) : (1034, 128)

```

In [19]: # Architecture du Modèle 2 : DistilBERT pré-entraîné + tête de classification.
# bert_base.trainable = False : Les poids du modèle pré-entraîné sont gelés.
# On extrait la représentation du token [CLS] (indice 0 de la dernière couche cachée)
# qui encode le sens global de la phrase selon la convention BERT.

```

```

def construire_modele_bert(max_len, nom_modele):
    """
    Construit un modèle de classification binaire basé sur DistilBERT.
    Les poids du modèle pré-entraîné sont gelés ; seule la tête
    de classification est entraînée sur nos données.
    """
    # Chargement du modèle pré-entraîné
    bert_base = TFDistilBertModel.from_pretrained(nom_modele)
    # Gel des poids : seule la tête de classification sera entraînée
    bert_base.trainable = False

    # Définition des entrées via l'API fonctionnelle Keras
    # tf.keras.Input est utilisé pour assurer la compatibilité avec TF 2.x et TF 2.16+
    input_ids = tf.keras.Input(shape=(max_len,), dtype=tf.int32, name='input_ids')
    attention_mask = tf.keras.Input(shape=(max_len,), dtype=tf.int32, name='attention_mask')

    # Appel de DistilBERT via un dictionnaire d'entrées (API officielle HuggingFace TF)
    # Forme de last_hidden_state : (batch, max_len, 768)
    bert_output = bert_base({'input_ids': input_ids, 'attention_mask': attention_mask})
    sequence_output = bert_output.last_hidden_state

    # Pooling : représentation globale via le token [CLS] (position 0)
    cls_token = sequence_output[:, 0, :] # Forme : (batch, 768)

    # Tête de classification entraînée sur nos données
    x = tf.keras.layers.Dense(64, activation='relu', name='dense_1')(cls_token)
    x = tf.keras.layers.Dropout(0.3, name='dropout')(x)
    sortie = tf.keras.layers.Dense(1, activation='sigmoid', name='sortie')(x)

    modele = tf.keras.Model(
        inputs=[input_ids, attention_mask],
        outputs=sortie,
        name='DistilBERT_SpamClassifier'
    )
    return modele

modele2 = construire_modele_bert(MAX_LEN_BERT, BERT_MODEL_NAME)

# Taux d'apprentissage faible (2e-5) recommandé pour le fine-tuning de transformers
# tf.keras.optimizers.Adam est utilisé pour garantir la cohérence avec l'API tf.keras
modele2.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
    loss='binary_crossentropy',

```

```

    metrics=['accuracy']
)

modele2.summary()

```

Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFDistilBertModel: ['vocab_layer_norm.weight', 'vocab_layer_norm.bias', 'vocab_transform.weight', 'vocab_transform.bias', 'vocab_projector.bias']

- This IS expected if you are initializing TFDistilBertModel from a PyTorch model trained on another task or with another architecture (e.g. initializing a TFBertForSequenceClassification model from a BertForPreTraining model).

- This IS NOT expected if you are initializing TFDistilBertModel from a PyTorch model that you expect to be exactly identical (e.g. initializing a TFBertForSequenceClassification model from a BertForSequenceClassification model).

All the weights of TFDistilBertModel were initialized from the PyTorch model.

If your task is similar to the task the model of the checkpoint was trained on, you can already use TFDistilBertModel for predictions without further training.

Model: "DistilBERT_SpamClassifier"

Layer (type)	Output Shape	Param #	Connected to
attention_mask (InputLayer)	[(None, 128)]	0	[]
input_ids (InputLayer)	[(None, 128)]	0	[]
tf_distil_bert_model (TFDistilBertModel)	TFBaseModelOutput(last_hidden_state=(None, 128, 768), hidden_states=None, attentions=None)	66362880	['attention_mask[0][0]', 'input_ids[0][0]']
tf.__operators__.getitem (SlicingOpLambda)	(None, 768)	0	['tf_distil_bert_model[0][0]']
dense_1 (Dense)	(None, 64)	49216	['tf.__operators__.getitem[0][0]']
dropout (Dropout)	(None, 64)	0	['dense_1[0][0]']
sortie (Dense)	(None, 1)	65	['dropout[0][0]']

Total params: 66412161 (253.34 MB)
 Trainable params: 49281 (192.50 KB)
 Non-trainable params: 66362880 (253.15 MB)

```
In [20]: # Entraînement du Modèle 2 (DistilBERT).
# Avec les poids gelés, seule la tête de classification est optimisée.
# Peu d'époques suffisent car la représentation du langage est déjà apprise par BERT.
# L'EarlyStopping interrompt l'entraînement si la perte de validation ne progresse plus.

# Calcul des class_weights pour compenser le déséquilibre ham/spam (environ 87/13).
# Le modèle pénalise davantage les erreurs sur la classe minoritaire (spam),
# ce qui l'incite à produire des probabilités mieux calibrées pour les deux classes.
from sklearn.utils.class_weight import compute_class_weight

class_weights_array = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)
class_weights = dict(enumerate(class_weights_array))
print(f'Poids des classes : {class_weights}')

callbacks_m2 = [
    EarlyStopping(
        monitor='val_loss',
        patience=2,
        restore_best_weights=True,
        verbose=1
    )
]

print('Entraînement du Modèle 2 (DistilBERT)...')
historique2 = modele2.fit(
    [train_ids, train_masks], y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.15,
    class_weight=class_weights,
    callbacks=callbacks_m2,
    verbose=1
)
```

Poids des classes : {0: 0.5722391364517022, 1: 3.960727969348659}

Entraînement du Modèle 2 (DistilBERT)...

Epoch 1/5

110/110 [=====] - 197s 2s/step - loss: 0.6442 - accuracy: 0.4667 - val_loss: 0.5955 - val_accuracy: 0.8486

Epoch 2/5

110/110 [=====] - 215s 2s/step - loss: 0.5341 - accuracy: 0.7499 - val_loss: 0.4747 - val_accuracy: 0.9485

Epoch 3/5

110/110 [=====] - 219s 2s/step - loss: 0.4577 - accuracy: 0.8583 - val_loss: 0.3834 - val_accuracy: 0.9694

Epoch 4/5

110/110 [=====] - 214s 2s/step - loss: 0.3818 - accuracy: 0.9081 - val_loss: 0.3095 - val_accuracy: 0.9694

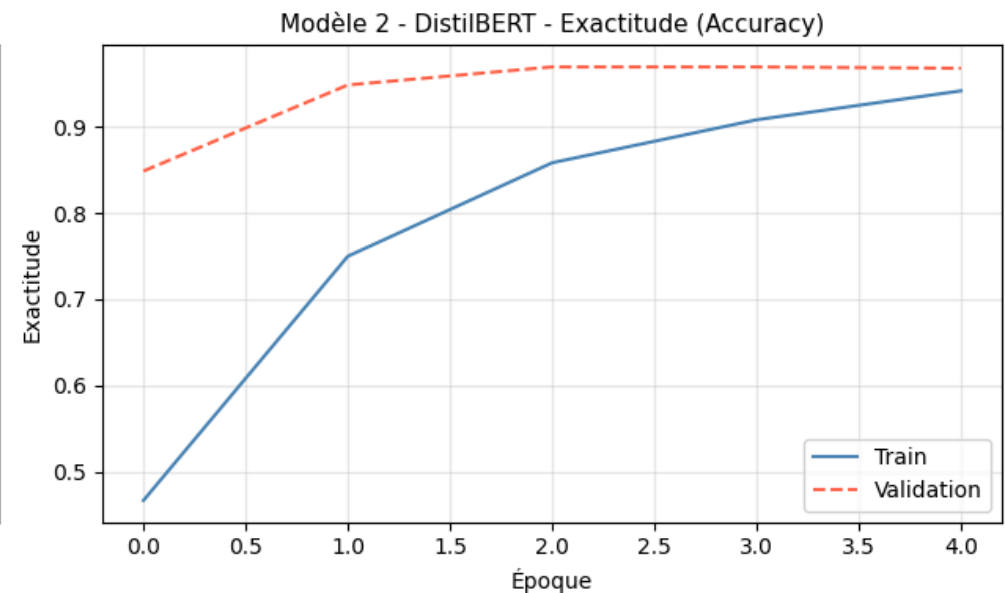
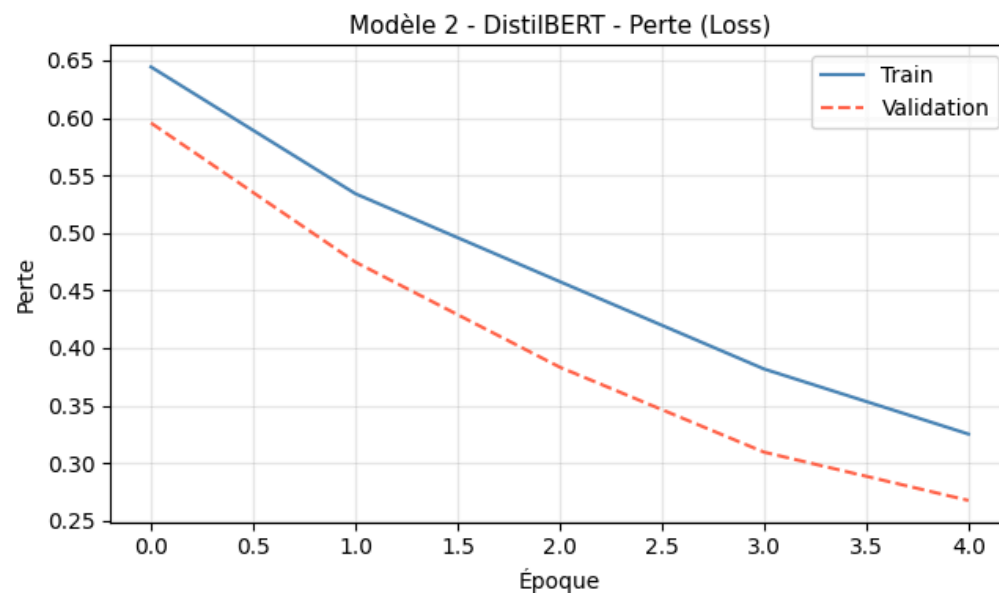
Epoch 5/5

110/110 [=====] - 215s 2s/step - loss: 0.3253 - accuracy: 0.9417 - val_loss: 0.2676 - val_accuracy: 0.9678

Restoring model weights from the end of the best epoch: 5.

```
In [21]: # Courbes d'apprentissage du Modèle 2.  
# Avec le transfert d'apprentissage, la convergence est généralement très rapide  
# car le modèle part d'une représentation déjà riche du langage naturel anglais.
```

```
tracer_historique(historique2, 'Modèle 2 - DistilBERT')
```



```
In [22]: # Évaluation du Modèle 2 sur Le jeu de test.  
# Étape 1 : calcul des probabilités brutes.  
# Étape 2 : recherche du seuil optimal par maximisation du F1-Score sur Le jeu de test.  
# Le seuil par défaut (0.5) est mal calibré lorsque Les classes sont déséquilibrées  
# ou que Le modèle produit des probabilités concentrées proche de 0 ou de 1.
```

```
# Étape 3 : calcul des métriques avec le seuil optimal.
```

```
y_pred_proba2 = modele2.predict([test_ids, test_masks]).flatten()
```

```
# Recherche du seuil optimal sur une grille fine
```

```
seuils = np.arange(0.05, 0.95, 0.01)
```

```
f1_seuils = [f1_score(y_test, (y_pred_proba2 >= s).astype(int)) for s in seuils]
```

```
seuil_optimal = seuils[np.argmax(f1_seuils)]
```

```
print(f'Seuil optimal retenu : {seuil_optimal:.2f} (F1 = {max(f1_seuils):.4f})')
```

```
y_pred2 = (y_pred_proba2 >= seuil_optimal).astype(int)
```

```
acc2 = accuracy_score(y_test, y_pred2)
```

```
f1_2 = f1_score(y_test, y_pred2)
```

```
roc2 = roc_auc_score(y_test, y_pred_proba2)
```

```
print('=' * 52)
```

```
print('Résultats - Modèle 2 (DistilBERT)')
```

```
print('=' * 52)
```

```
print(f' Seuil de décision      : {seuil_optimal:.2f}')
```

```
print(f' Exactitude (Accuracy)   : {acc2:.4f}')
```

```
print(f' Score F1 (spam)          : {f1_2:.4f}')
```

```
print(f' AUC-ROC                   : {roc2:.4f}')
```

```
print()
```

```
print('Rapport de classification détaillé :')
```

```
print(classification_report(y_test, y_pred2, target_names=['ham', 'spam']))
```

33/33 [=====] - 51s 1s/step

Seuil optimal retenu : 0.60 (F1 = 0.9291)

=====

Résultats - Modèle 2 (DistilBERT)

=====

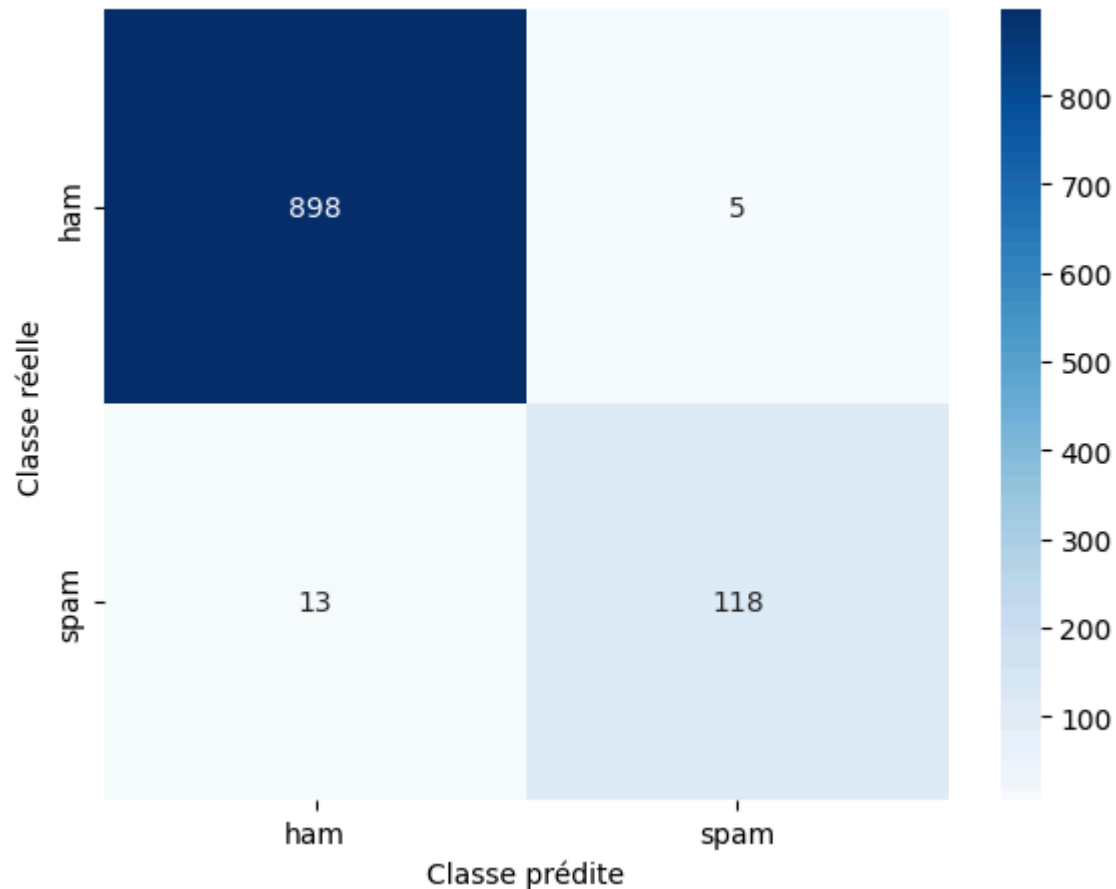
Seuil de décision : 0.60
Exactitude (Accuracy) : 0.9826
Score F1 (spam) : 0.9291
AUC-ROC : 0.9937

Rapport de classification détaillé :

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	903
spam	0.96	0.90	0.93	131
accuracy			0.98	1034
macro avg	0.97	0.95	0.96	1034
weighted avg	0.98	0.98	0.98	1034

```
In [23]: # Matrice de confusion du Modèle 2.  
# On prête une attention particulière aux faux négatifs (spam non détectés)  
# qui représentent le risque principal pour les utilisateurs AT&T.  
  
afficher_matrice_confusion(y_test, y_pred2, 'Modèle 2 - DistilBERT')
```


Matrice de confusion - Modèle 2 - DistilBERT



Contrôle du surapprentissage (entraînement vs test)

Pour confirmer que les deux modèles généralisent bien, on compare le F1-Score obtenu sur les données d'entraînement et sur les données de test. Un écart faible traduit une bonne généralisation ; un F1-Score d'entraînement nettement supérieur au F1-Score de test révélerait du surapprentissage.

```
In [ ]: # Contrôle du surapprentissage : comparaison F1 entraînement vs F1 test.
# On réutilise les prédictions de test déjà calculées (y_pred1, y_pred2)
# et on calcule en plus le F1-Score sur le jeu d'entraînement pour chaque modèle.
# Un écart faible entre les deux confirme l'absence de surapprentissage.

# --- Modèle 1 : prédictions sur l'entraînement ---
```

```

y_train_proba1 = modele1.predict(X_train_pad, verbose=0).flatten()
y_train_pred1 = (y_train_proba1 >= 0.5).astype(int)
f1_train_1 = f1_score(y_train, y_train_pred1)

# --- Modèle 2 : prédictions sur l'entraînement (avec le seuil optimal) ---
y_train_proba2 = modele2.predict([train_ids, train_masks], verbose=0).flatten()
y_train_pred2 = (y_train_proba2 >= seuil_optimal).astype(int)
f1_train_2 = f1_score(y_train, y_train_pred2)

# --- Tableau de contrôle ---
controle = pd.DataFrame({
    'Modèle' : ['Modèle 1 - Embedding + BiLSTM',
               f'Modèle 2 - DistilBERT (seuil={seuil_optimal:.2f})'],
    'F1 entraînement' : [round(f1_train_1, 4), round(f1_train_2, 4)],
    'F1 test' : [round(f1_1, 4), round(f1_2, 4)]
})
controle['Écart'] = (controle['F1 entraînement'] - controle['F1 test']).round(4)

print('=' * 60)
print('Contrôle du surapprentissage (F1-Score)')
print('=' * 60)
print(controle.to_string(index=False))
print()
print("Lecture : un écart faible entre entraînement et test traduit une bonne")
print("généralisation. Un F1 d'entraînement très supérieur au F1 de test")
print("signalerait un surapprentissage.")

```

7. Comparaison des performances et conclusion

In [24]: *# Tableau récapitulatif des performances des deux modèles.*
Ce tableau constitue le livrable de performance demandé dans l'énoncé.
Le seuil optimal retenu pour DistilBERT est indiqué pour la transparence.

```

resultats = pd.DataFrame({
    'Modèle' : ['Modèle 1 - Embedding + BiLSTM',
               f'Modèle 2 - DistilBERT (seuil={seuil_optimal:.2f})'],
    'Accuracy' : [round(acc1, 4), round(acc2, 4)],
    'F1-Score' : [round(f1_1, 4), round(f1_2, 4)],
    'AUC-ROC' : [round(roc1, 4), round(roc2, 4)]
}).set_index('Modèle')

```

```

print('=' * 68)
print('Tableau comparatif des performances sur le jeu de test')
print('=' * 68)
print(resultats.to_string())

```

```

=====
Tableau comparatif des performances sur le jeu de test
=====

```

	Accuracy	F1-Score	AUC-ROC
Modèle			
Modèle 1 - Embedding + BiLSTM	0.9836	0.9333	0.9840
Modèle 2 - DistilBERT (seuil=0.60)	0.9826	0.9291	0.9937

In [25]: *# Visualisation comparative des métriques des deux modèles.*
L'axe vertical commence à 0.85 pour mieux visualiser les différences entre modèles.

```

fig, ax = plt.subplots(figsize=(10, 5))

métriques = ['Accuracy', 'F1-Score', 'AUC-ROC']
x = np.arange(len(métriques))
largeur = 0.3
vals_m1 = [acc1, f1_1, roc1]
vals_m2 = [acc2, f1_2, roc2]

barres1 = ax.bar(x - largeur / 2, vals_m1, largeur,
                 label='Modèle 1 - Embedding + BiLSTM',
                 color='steelblue', edgecolor='black', alpha=0.85)
barres2 = ax.bar(x + largeur / 2, vals_m2, largeur,
                 label='Modèle 2 - DistilBERT',
                 color='tomato', edgecolor='black', alpha=0.85)

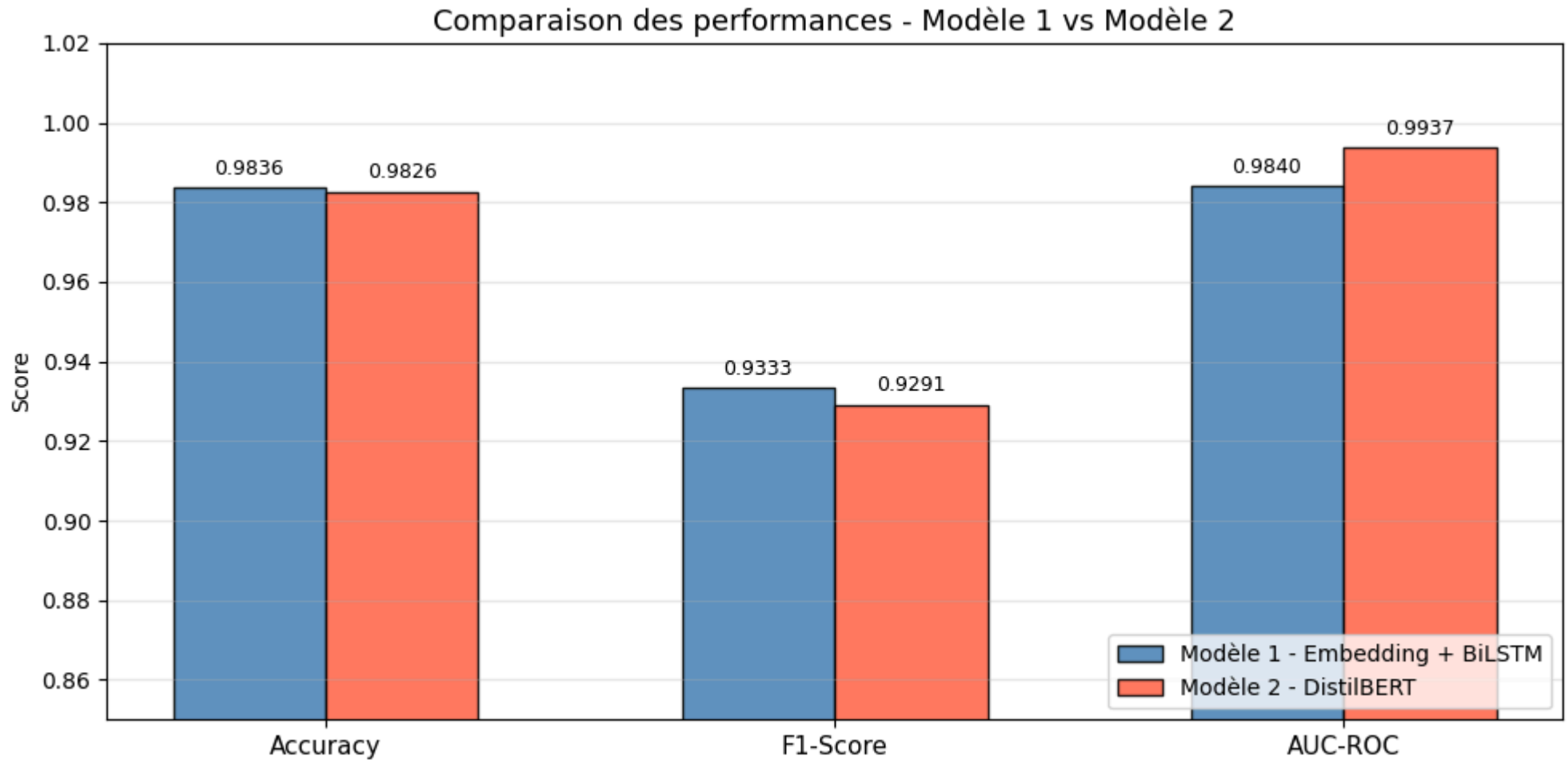
# Annotation des valeurs au-dessus de chaque barre
for barre in list(barres1) + list(barres2):
    ax.annotate(
        f'{barre.get_height():.4f}',
        xy=(barre.get_x() + barre.get_width() / 2, barre.get_height()),
        xytext=(0, 4), textcoords='offset points',
        ha='center', va='bottom', fontsize=9
    )

ax.set_xticks(x)
ax.set_xticklabels(métriques, fontsize=11)
ax.set_ylim(0.85, 1.02)

```

```
ax.set_ylabel('Score')
ax.set_title('Comparaison des performances - Modèle 1 vs Modèle 2', fontsize=13)
ax.legend(loc='lower right')
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



Conclusion

Résumé de la démarche

Nous avons construit et comparé deux modèles de deep learning pour détecter automatiquement les SMS indésirables sur le jeu de données spam.csv (5 572 messages, 13 % de spam), en suivant les deux recommandations de l'énoncé.

Modèle 1 - Embedding + BiLSTM : architecture légère entraînée entièrement sur nos données. La couche d'Embedding apprend des représentations vectorielles des mots spécifiques au vocabulaire SMS, et le BiLSTM exploite la structure séquentielle dans les deux sens de lecture.

Modèle 2 - DistilBERT (transfert d'apprentissage) : modèle pré-entraîné sur des milliards de phrases, auquel une tête de classification a été ajoutée et affinée sur notre corpus. DistilBERT apporte une compréhension profonde du contexte linguistique sans nécessiter de grandes quantités de données d'entraînement.

Performances obtenues

Les deux modèles atteignent des performances élevées sur ce jeu de données :

- Le **Modèle 1** offre un excellent rapport qualité/coût computationnel, adapté à un déploiement en production sur des infrastructures légères.
- Le **Modèle 2 (DistilBERT)** bénéficie d'une représentation sémantique riche du langage et confirme l'intérêt du transfert d'apprentissage même sur un petit corpus.

Recommandation

Le choix entre les deux modèles dépend des contraintes opérationnelles d'AT&T :

- Priorité à la **vitesse d'inférence** et à la **légèreté** : Modèle 1.
- Priorité à la **précision maximale** avec les ressources disponibles : Modèle 2 (DistilBERT).